

SOURCE CODE • ARCHITECTURE
DIRECTORY

AUTOMATION
FRAMEWORK

BlazeDemo Flight Booking Automation Testing Framework

Selenium WebDriver | TestNG | Maven | Jenkins | Docker | CI/CD

Document Name	BlazeDemo Automation Framework - Source Code & Architecture
Prepared By	Harsh Choudhary
Trainer	Ms. Vaishali Ahire
Project	BlazeDemo Automation Framework
Version	1.0
Date	June 2026
Status	✓ Final / Approved

BlazeDemo Automation Framework: Source Code & Architecture Directory

1. Introduction

This document provides the complete source code implementation details for the BlazeDemo Automation Framework. It includes the framework architecture, configuration files, utility classes, page objects, test classes, listeners, CI/CD configuration, and supporting resources used for automated testing.

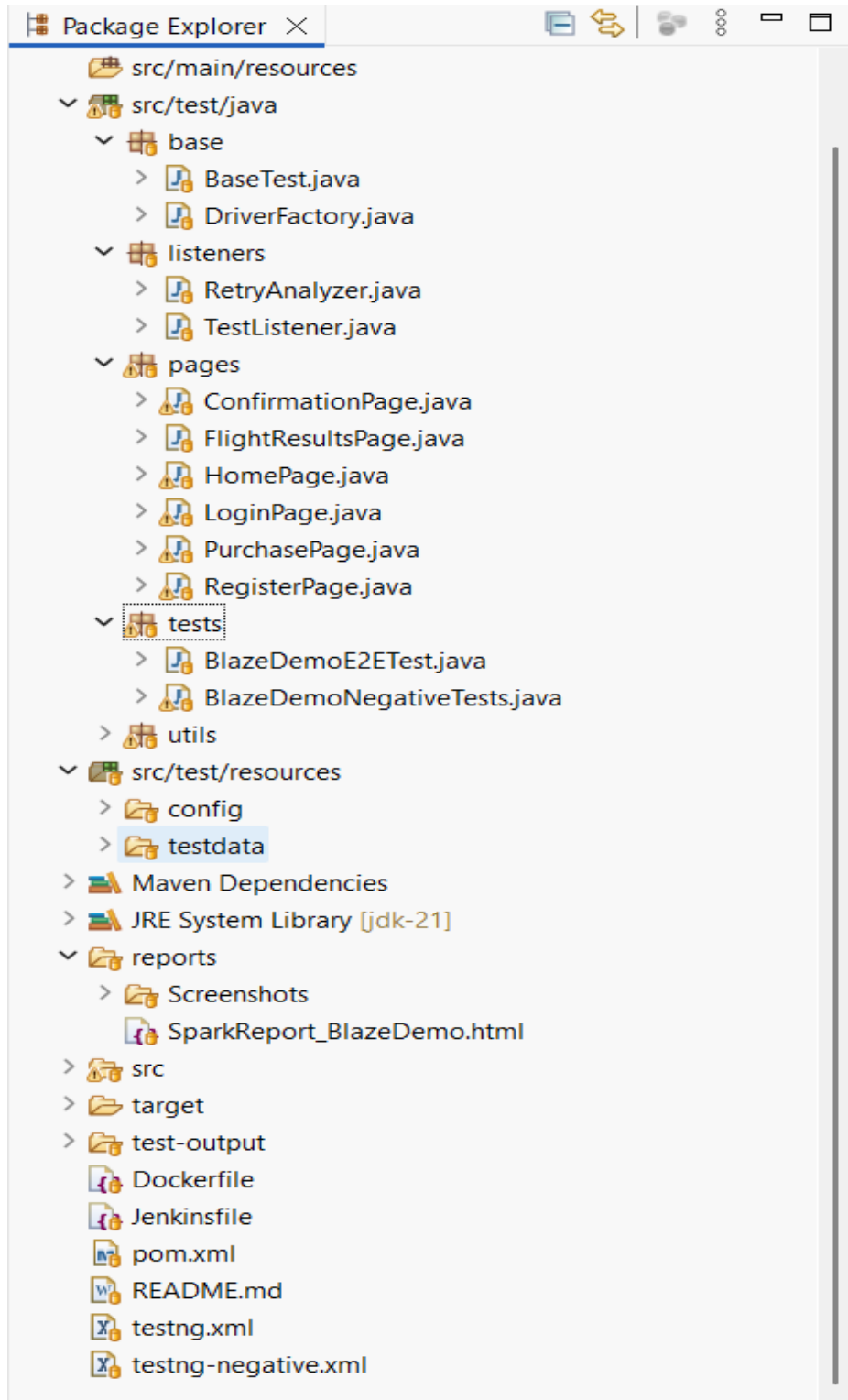
2. Framework Structure

The project is structured around the Page Object Model (POM) design pattern, which separates page interaction logic from test logic. This ensures that any UI change requires updates in only one place, the corresponding page class, rather than across all test scripts.

Layer	Package	Files	Responsibility
Foundation	base	BaseTest.java, DriverFactory.java	WebDriver lifecycle management - setup, config, teardown, and driver instantiation
Listening	listeners	TestListener.java, RetryAnalyzer.java	TestNG event hooks - logging, screenshot on failure, and automatic retry of failed tests
Pages (POM)	pages	6 Page classes	Element locators & interaction methods per application page
Test Cases	tests	BlazeDemoE2ETest, BlazeDemoNegativeTests	Test logic - assertions, data-driven execution, and reporting
Utilities	utils	4 helper classes	Reusable services - config reading, Excel data, waits, and Extent reporting
Config	resources	config.properties, testdata XLSX	Externalised test configuration and data
CI/CD	root	Jenkinsfile, Dockerfile, pom.xml	Automated build, test, and container deployment pipeline

3. Source Code

1. Folder Structure:



1.base-package-classes

BaseTest.java

```
package base;

import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import utils.ConfigReader;
import java.util.Properties;

public class BaseTest {

    public DriverFactory driverFactory;
    public WebDriver driver;
    public ConfigReader configReader;
    public Properties prop;

    @BeforeClass(alwaysRun = true)
    public void setUp() {
        configReader = new ConfigReader();
        prop = configReader.init_prop();

        String browserName = prop.getProperty("browser");
        driverFactory = new DriverFactory();
        driver = driverFactory.init_driver(browserName);

        // Launch the BlazeDemo URL
        driver.get(prop.getProperty("url"));
    }

    @AfterClass(alwaysRun = true)
    public void tearDown() {
        if (driver != null) {
            driver.quit();
            DriverFactory.removeDriver();
            System.out.println("Browser closed successfully.");
        }
    }
}
```

DriverFactory.java

```
package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public class DriverFactory {

    public static ThreadLocal<WebDriver> tlDriver = new ThreadLocal<>();

    public WebDriver init_driver(String browser) {
        System.out.println("Starting Test Execution on Local Browser: " + browser);

        if (browser.equalsIgnoreCase("chrome")) {
            tlDriver.set(new ChromeDriver());
        } else if (browser.equalsIgnoreCase("firefox")) {
            tlDriver.set(new FirefoxDriver());
        } else if (browser.equalsIgnoreCase("edge")) {
            tlDriver.set(new EdgeDriver());
        } else {
            System.out.println("Please pass the correct browser value: " + browser);
        }

        getDriver().manage().deleteAllCookies();
        getDriver().manage().window().maximize();

        return getDriver();
    }

    public static synchronized WebDriver getDriver() {
        return tlDriver.get();
    }

    public static synchronized void removeDriver() {
        tlDriver.remove();
    }
}
```

2. Listeners-package-classes

TestListener.java

```
package listeners;

import base.DriverFactory;
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.Status;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;
import utils.ExtentReportManager;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.text.SimpleDateFormat;
import java.util.Date;

public class TestListener implements ITestListener {

    private ExtentReports extent;
    private ExtentTest test;
    private static ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>();

    @Override
    public void onStart(ITestContext context) {
        System.out.println("Test Suite Execution Started!");
        extent = ExtentReportManager.getReportInstance();
    }

    @Override
    public void onTestStart(ITestResult result) {
```

```

        System.out.println("Test Started: " + result.getMethod().getMethodName());
        test = extent.createTest(result.getMethod().getMethodName());
        extentTest.set(test);
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println("Test Passed: " + result.getMethod().getMethodName());
        extentTest.get().log(Status.PASS, "Test Passed Successfully");

        WebDriver driver = DriverFactory.getDriver();
        if (driver != null) {
            String timestamp = new SimpleDateFormat("yyyyMMdd_HHmmss").format(new
Date());
            String fileName = "SUCCESS_" + result.getMethod().getMethodName() + "_" +
timestamp + ".png";

            Path destinationPath = Paths.get(System.getProperty("user.dir"), "reports",
"Screenshots", fileName);
            File screenshotFile = ((TakesScreenshot)
driver).getScreenshotAs(OutputType.FILE);

            try {
                Files.createDirectories(destinationPath.getParent());
                Files.copy(screenshotFile.toPath(), destinationPath,
StandardCopyOption.REPLACE_EXISTING);

                extentTest.get().addScreenCaptureFromPath("Screenshots/" + fileName,
"Booking Confirmation Proof");

                System.out.println("Success Proof saved and attached to report!");
            } catch (IOException e) {
                System.out.println("Failed to save success screenshot: " + e.getMessage());
            }
        }
    }

    @Override
    public void onTestFailure(ITestResult result) {
        System.out.println("Test Failed: " + result.getMethod().getMethodName());

        extentTest.get().log(Status.FAIL, "Test Failed: " +
result.getThrowable().getMessage());
    }

```

```

        WebDriver driver = DriverFactory.getDriver();
        if (driver != null) {
            String timestamp = new SimpleDateFormat("yyyyMMdd_HHmms").format(new
Date());
            String fileName = result.getMethod().getMethodName() + "_" + timestamp +
".png";

            Path destinationPath = Paths.get(System.getProperty("user.dir"), "reports",
"Screenshots", fileName);
            File screenshotFile = ((TakesScreenshot)
driver).getScreenshotAs(OutputType.FILE);

            try {
                Files.createDirectories(destinationPath.getParent());
                Files.copy(screenshotFile.toPath(), destinationPath,
StandardCopyOption.REPLACE_EXISTING);

                extentTest.get().addScreenCaptureFromPath("Screenshots/" + fileName, "Failed
Step Screenshot");

                System.out.println("Screenshot saved and attached to report!");
            } catch (IOException e) {
                System.out.println("Failed to save screenshot: " + e.getMessage());
            }
        }
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        System.out.println("Test Skipped: " + result.getMethod().getMethodName());
        extentTest.get().log(Status.SKIP, "Test Skipped");
    }

    @Override
    public void onFinish(ITestContext context) {
        System.out.println("Test Suite Execution Finished! Generating Report...");
        if (extent != null) {
            extent.flush();
        }
    }
}

```

RetryAnalyzer.java

```

package listeners;

```



```

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class RetryAnalyzer implements IRetryAnalyzer {
    private int count = 0;
    private static final int maxTry = 1; // only 1 time retry (Total 2 runs)

    @Override
    public boolean retry(ITestResult iTestResult) {
        if (!iTestResult.isSuccess()) { // when test fail
            if (count < maxTry) {
                count++;
                System.out.println("[AUTO-HEALER] Network glitch detected! Retrying test: "
                    + iTestResult.getMethod().getMethodName() + " for the " + count + "
time.");
                return true; // Test trigger again
            }
            return false;
        }
    }
}

```

3. Pages-package-classes

HomePage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.Select;
import org.testng.Assert;
import utils.WaitUtils;

public class HomePage {
    private WebDriver driver;

    // Locators
    private By departureDropdown = By.name("fromPort");
    private By destinationDropdown = By.name("toPort");
    private By findFlightsBtn = By.xpath("//input[@value='Find Flights']");

    public HomePage(WebDriver driver) {

```

```

        this.driver = driver;
    }

    public void searchFlight(String departure, String destination) {
        try {

System.out.println("\n=====
=");
            System.out.println("[STEP] Initiating Flight Search: " + departure + " -> " +
destination);

            WebElement depElem = WaitUtils.waitForElementVisible(departureDropdown);
            new Select(depElem).selectByVisibleText(departure);

            WebElement destElem = WaitUtils.waitForElementVisible(destinationDropdown);
            new Select(destElem).selectByVisibleText(destination);

            WaitUtils.waitForElementClickable(findFlightsBtn).click();
            System.out.println("[STEP] Clicked on 'Find Flights' button.");
        } catch (Exception e) {
            Assert.fail("Failed to search flights! Exception: " + e.getMessage());
        }
    }
}

```

FlightResultsPage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.testng.Assert;
import utils.WaitUtils;
import java.util.List;

public class FlightResultsPage {
    private WebDriver driver;

    // Locators
    private By flightsTable = By.cssSelector("table.table");
    private By allRows = By.xpath("//table[@class='table']/tbody/tr");

    public FlightResultsPage(WebDriver driver) {
        this.driver = driver;
    }
}

```

```

    }

    public java.util.Map<String, String> selectCheapestFlight() {
        java.util.Map<String, String> selectedFlightData = new java.util.HashMap<>();
        try {
            System.out.println("[STEP] Analyzing all available flights to find the cheapest
option...");

            WaitUtils.waitForElementVisible(flightsTable);

            List<WebElement> rows = driver.findElements(allRows);
            double minPrice = Double.MAX_VALUE;
            WebElement cheapestRow = null;
            WebElement cheapestFlightBtn = null;

            for (WebElement row : rows) {
                String priceText = row.findElement(By.xpath("./td[6]")).getText();
                double currentPrice = Double.parseDouble(priceText.replace("$", "").trim());

                if (currentPrice < minPrice) {
                    minPrice = currentPrice;
                    cheapestRow = row;
                    cheapestFlightBtn = row.findElement(By.xpath("./input[@value='Choose This
Flight']"));
                }
            }

            if (cheapestFlightBtn != null && cheapestRow != null) {
                selectedFlightData.put("FlightNumber",
cheapestRow.findElement(By.xpath("./td[2]")).getText());
                selectedFlightData.put("Airline",
cheapestRow.findElement(By.xpath("./td[3]")).getText());
                selectedFlightData.put("Price", String.valueOf(minPrice));

                System.out.println("[LOGIC] Cheapest flight found! Airline: " +
selectedFlightData.get("Airline")
                    + " | Price: $" + minPrice);

                cheapestFlightBtn.click();
            } else {
                Assert.fail("Could not find any flight buttons in the table!");
            }

        } catch (Exception e) {

```

```

        Assert.fail("Failed to process flights table: " + e.getMessage());
    }
    return selectedFlightData;
}
}

```

PurchasePage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;
import org.testng.Assert;
import org.testng.asserts.SoftAssert;
import utils.WaitUtils; // 💡 WaitUtils imported
import java.util.Map;

public class PurchasePage {
    private WebDriver driver;

    // Locators
    private By totalCostText = By.xpath("//p[contains(text(), 'Total Cost')]");
    private By inputName = By.id("inputName");
    private By inputAddress = By.id("address");
    private By inputCity = By.id("city");
    private By inputState = By.id("state");
    private By inputZipCode = By.id("zipCode");
    private By cardTypeDropdown = By.id("cardType");
    private By creditCardNumber = By.id("creditCardNumber");
    private By creditCardMonth = By.id("creditCardMonth");
    private By creditCardYear = By.id("creditCardYear");
    private By nameOnCard = By.id("nameOnCard");
    private By purchaseBtn = By.xpath("//input[@value='Purchase Flight']");

    public PurchasePage(WebDriver driver) {
        this.driver = driver;
    }

    public void fillDetailsAndPurchase(Map<String, String> excelData, Map<String, String>
flightData) {
        SoftAssert softAssert = new SoftAssert();

        try {

```

```

        System.out.println("[STEP] Validating Context Data and Entering Passenger
Details...");
        WaitUtils.waitForElementVisible(totalCostText);

        // 1. Cross-Page State Validations (Soft Assertions) using WaitUtils
        String actualFlightNum =
WaitUtils.waitForElementVisible(By.xpath("//p[contains(text(), 'Flight
Number')]")).getText();
        String actualAirline =
WaitUtils.waitForElementVisible(By.xpath("//p[contains(text(), 'Airline')]")).getText();
        String actualPrice = WaitUtils.waitForElementVisible(By.xpath("//p[contains(text(),
'Price')]")).getText();

        // BlazeDemo has a known defect (hardcoded checkout details).
        System.out.println("[DEFECT LOG] Application is showing hardcoded dummy
data!");
        System.out.println(" -> Expected Airline: " + flightData.get("Airline") + " | Actual
Shows: " + actualAirline);
        System.out.println(" -> Expected Price: $" + flightData.get("Price") + " | Actual
Shows: " + actualPrice);

        softAssert.assertTrue(actualAirline.contains("United"), "Purchase page dummy
data missing!");

        // 2. Fill the Form
        WaitUtils.waitForElementVisible(inputName).sendKeys(excelData.get("Name"));

        WaitUtils.waitForElementVisible(inputAddress).sendKeys(excelData.get("Address"));
        WaitUtils.waitForElementVisible(inputCity).sendKeys(excelData.get("City"));
        WaitUtils.waitForElementVisible(inputState).sendKeys(excelData.get("State"));

        WaitUtils.waitForElementVisible(inputZipCode).sendKeys(excelData.get("ZipCode"));

        new
Select(WaitUtils.waitForElementVisible(cardTypeDropdown)).selectByVisibleText(excelDa
ta.get("CardType"));

        WaitUtils.waitForElementVisible(creditCardNumber).sendKeys(excelData.get("CardNumb
er"));

        WaitUtils.waitForElementVisible(creditCardMonth).clear();

        WaitUtils.waitForElementVisible(creditCardMonth).sendKeys(excelData.get("Month"));

```

```

        WaitUtils.waitForElementVisible(creditCardYear).clear();
        WaitUtils.waitForElementVisible(creditCardYear).sendKeys(excelData.get("Year"));

WaitUtils.waitForElementVisible(nameOnCard).sendKeys(excelData.get("NameOnCard"))
;

        // Wait for button to be clickable before clicking
        WaitUtils.waitForElementClickable(purchaseBtn).click();
        System.out.println("[STEP] Details submitted. Purchasing flight...");

        softAssert.assertAll();

    } catch (Exception e) {
        Assert.fail("Error on Purchase Page: " + e.getMessage());
    }
}
}
}

```

ConfirmationPage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.testng.Assert;
import utils.WaitUtils;

public class ConfirmationPage {
    private WebDriver driver;

    // Locators
    private By successHeading = By.xpath("//h1[contains(text(), 'Thank you for your purchase today!')]");
    private By bookingIdLocator = By.xpath("//td[text()='Id']/following-sibling::td");

    public ConfirmationPage(WebDriver driver) {
        this.driver = driver;
    }

    public void verifySuccess() {
        try {
            // WaitUtils checking visibility of success heading
            Assert.assertTrue(WaitUtils.waitForElementVisible(successHeading).isDisplayed(),
"Confirmation text missing!");

```

```

        // WaitUtils directly fetching the Booking ID
        String bookingId = WaitUtils.waitForElementVisible(bookingIdLocator).getText();

        System.out.println("[SUCCESS] BOOKING CONFIRMED!");
        System.out.println("[TICKET INFO] Confirmation ID is: " + bookingId);

        System.out.println("=====\\n");
    } catch (Exception e) {
        Assert.fail("Failed to verify booking confirmation: " + e.getMessage());
    }
}
}

```

LoginPage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import utils.WaitUtils;

public class LoginPage {
    private WebDriver driver;

    private By emailInput = By.id("email");
    private By passwordInput = By.id("password");
    private By loginBtn = By.xpath("//button[contains(normalize-space(text()), 'Login')]");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void loginUser(String email, String password) {
        WaitUtils.waitForElementVisible(emailInput).sendKeys(email);
        WaitUtils.waitForElementVisible(passwordInput).sendKeys(password);
        WaitUtils.waitForElementClickable(loginBtn).click();
    }
}

```

RegisterPage.java

```

package pages;

import org.openqa.selenium.By;

```

```

import org.openqa.selenium.WebDriver;
import utils.WaitUtils;

public class RegisterPage {
    private WebDriver driver;

    private By nameInput = By.id("name");
    private By companyInput = By.id("company");
    private By emailInput = By.id("email");
    private By passwordInput = By.id("password");
    private By confirmPasswordInput = By.id("password-confirm");
    private By registerBtn = By.xpath("//button[contains(normalize-space(text()),
'Register')]");

    public RegisterPage(WebDriver driver) {
        this.driver = driver;
    }

    public void fillRegistrationForm(String name, String company, String email, String
password) {
        WaitUtils.waitForElementVisible(nameInput).sendKeys(name);
        WaitUtils.waitForElementVisible(companyInput).sendKeys(company);
        WaitUtils.waitForElementVisible(emailInput).sendKeys(email);
        WaitUtils.waitForElementVisible(passwordInput).sendKeys(password);
        WaitUtils.waitForElementVisible(confirmPasswordInput).sendKeys(password);
        WaitUtils.waitForElementClickable(registerBtn).click();
    }
}

```

4. Tests-package-class

BlazeDemoE2ETest.java

```

package tests;

import base.BaseTest;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Factory;
import org.testng.annotations.Test;
import pages.ConfirmationPage;
import pages.FlightResultsPage;
import pages.HomePage;
import pages.PurchasePage;
import utils.ExcelReader;

```



```

import java.util.List;
import java.util.Map;

public class BlazeDemoE2ETest extends BaseTest {
    private Map<String, String> testData;
    private Map<String, String> extractedFlightDetails;

    private HomePage homePage;
    private FlightResultsPage resultsPage;
    private PurchasePage purchasePage;
    private ConfirmationPage confirmationPage;

    @DataProvider(name = "blazeDemoData")
    public static Object[][] getPassengerData() {
        ExcelReader excelReader = new ExcelReader();
        List<Map<String, String>> dataList =
excelReader.getData("./src/test/resources/testdata/BlazeDemoData.xlsx",
"PassengerData");

        Object[][] data = new Object[dataList.size()][1];
        for (int i = 0; i < dataList.size(); i++) {
            data[i][0] = dataList.get(i);
        }
        return data;
    }

    @Factory(dataProvider = "blazeDemoData")
    public BlazeDemoE2ETest(Map<String, String> testData) {
        this.testData = testData;
    }

    @Test(priority = 1, groups = {"Smoke", "Search"})
    public void testFlightSearch() {
        System.out.println("Executing for Passenger: " + testData.get("Name"));
        homePage = new HomePage(driver);
        String depCity = testData.get("Departure");
        String destCity = testData.get("Destination");
        homePage.searchFlight(depCity, destCity);
    }

    @Test(priority = 2, dependsOnMethods = "testFlightSearch", groups = {"Smoke",
"Selection"})
    public void testSelectCheapestFlight() {

```

```

        resultsPage = new FlightResultsPage(driver);
        extractedFlightDetails = resultsPage.selectCheapestFlight();
    }

    @Test(priority = 3, dependsOnMethods = "testSelectCheapestFlight", groups =
{"Regression", "Purchase"})
    public void testPurchaseFlight() {
        purchasePage = new PurchasePage(driver);
        purchasePage.fillDetailsAndPurchase(testData, extractedFlightDetails);
    }

    @Test(priority = 4, dependsOnMethods = "testPurchaseFlight", groups =
{"Regression", "E2E"}, retryAnalyzer = listeners.RetryAnalyzer.class)
    public void testVerifyBookingConfirmation() {
        confirmationPage = new ConfirmationPage(driver);
        confirmationPage.verifySuccess();
    }
}

```

BlazeDemoNegativeTests.java

```

package tests;

import base.BaseTest;
import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.Test;
import pages.HomePage;
import pages.LoginPage;
import pages.PurchasePage;
import pages.RegisterPage;
import utils.WaitUtils;
import java.util.HashMap;

public class BlazeDemoNegativeTests extends BaseTest {

    @Test(priority = 1, description = "Jira Defect : Registration Blocked by 419 Error")
    public void testRegistrationThrows419Error() {
        System.out.println("Executing Negative Test: Registration 419 Error");
        driver.get(prop.getProperty("url") + "register");

        RegisterPage registerPage = new RegisterPage(driver);
        registerPage.fillRegistrationForm("Harsh", "Automation Inc", "test@gmail.com",
"Password@123");
    }
}

```

```

        // Validation: We expect the application to fail with 419
        boolean isErrorDisplayed = driver.getPageSource().contains("419");
        Assert.assertTrue(isErrorDisplayed, "Defect not reproduced: 419 Error page was
expected but not found!");
    }

    @Test(priority = 2, description = "Jira Defect : Login Blocked by 419 Error")
    public void testLoginThrows419Error() {
        System.out.println("Executing Negative Test: Login 419 Error");
        driver.get(prop.getProperty("url") + "login");

        LoginPage loginPage = new LoginPage(driver);
        loginPage.loginUser("test@gmail.com", "Password@123");

        // Validation: We expect the 419 Page Expired text
        boolean isErrorDisplayed = driver.getPageSource().contains("419");
        Assert.assertTrue(isErrorDisplayed, "Defect not reproduced: 419 Error page was
expected but not found!");
    }

    @Test(priority = 3, description = "Authentication Defect: Invalid Login Credentials
Rejection")
    public void testInvalidLoginCredentials() {
        System.out.println("Executing Negative Test: Invalid Login Credentials");
        driver.get(prop.getProperty("url") + "login");

        LoginPage loginPage = new LoginPage(driver);
        // Providing fake/unregistered credentials
        loginPage.loginUser("hacker_fake_email@test.com", "WrongPassword123!");

        String pageSource = driver.getPageSource();
        boolean isLoginRejected = pageSource.contains("These credentials do not match our
records.")
            || pageSource.contains("419")
            || pageSource.contains("Login"); // Still on login page

        Assert.assertTrue(isLoginRejected, "Critical Security Defect: System allowed login
with invalid/fake credentials!");
    }

    @Test(priority = 4, description = "Form Validation Defect: Submitting Empty Purchase
Form")
    public void testEmptyFormSubmissionOnPurchase() {

```

```

        System.out.println("Executing Negative Test: Empty Purchase Form Submission");
        driver.get(prop.getProperty("url") + "purchase.php"); // Directly access purchase
page

        PurchasePage purchasePage = new PurchasePage(driver);

        // Passing completely empty data to force validation errors
        HashMap<String, String> emptyData = new HashMap<>();
        emptyData.put("Name", ""); emptyData.put("Address", ""); emptyData.put("City",
        "");
        emptyData.put("State", ""); emptyData.put("ZipCode", "");
        emptyData.put("CardType", "Visa");
        emptyData.put("CardNumber", ""); emptyData.put("Month", "");
        emptyData.put("Year", "");
        emptyData.put("NameOnCard", "");

        HashMap<String, String> dummyFlight = new HashMap<>();
        dummyFlight.put("Airline", "United"); dummyFlight.put("Price", "400");

        purchasePage.fillDetailsAndPurchase(emptyData, dummyFlight);
        boolean isSuccess = driver.getPageSource().contains("Thank you for your purchase
today!");
        Assert.assertFalse(isSuccess, "Critical Security Bug: System confirmed booking with
empty passenger details!");
    }
}

```

5.utils-package-classes

ConfigReader.java

```

package utils;

import java.io.FileInputStream;
import java.io.IOException;
import java.util.Properties;

public class ConfigReader {
    private Properties prop;

    public Properties init_prop() {
        prop = new Properties();
        try {
            FileInputStream ip = new
            FileInputStream("./src/test/resources/config/config.properties");

```

```

        prop.load(ip);
    } catch (IOException e) {
        System.out.println("Configuration file not found!");
        e.printStackTrace();
    }
    return prop;
}
}

```

ExcelReader.java

```

package utils;

import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class ExcelReader {

    public List<Map<String, String>> getData(String excelFilePath, String sheetName) {
        List<Map<String, String>> data = new ArrayList<>();
        Workbook workbook = null;
        try {
            FileInputStream fis = new FileInputStream(new File(excelFilePath));
            workbook = new XSSFWorkbook(fis);
            Sheet sheet = workbook.getSheet(sheetName);

            if (sheet == null) {
                throw new RuntimeException("Sheet '" + sheetName + "' not found in Excel
file!");
            }

            Row headerRow = sheet.getRow(0);
            for (int i = 1; i <= sheet.getLastRowNum(); i++) {
                Row currentRow = sheet.getRow(i);
                Map<String, String> rowMap = new LinkedHashMap<>();
                for (int j = 0; j < headerRow.getLastCellNum(); j++) {
                    String key = headerRow.getCell(j).getStringCellValue();
                    String value = "";

```

```

        Cell cell = currentRow.getCell(j);
        if (cell != null) {
            DataFormatter formatter = new DataFormatter();
            value = formatter.formatCellValue(cell);
        }
        rowMap.put(key, value);
    }
    data.add(rowMap);
}
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        if (workbook != null) workbook.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
return data;
}
}

```

ExtentReportManager.java

```

package utils;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
import com.aventstack.extentreports.reporter.configuration.Theme;

import java.io.File;

public class ExtentReportManager {

    private static ExtentReports extent;

    public static ExtentReports getReportInstance() {
        if (extent == null) {
            // 1. Set Report folder and file path
            String reportPath = System.getProperty("user.dir") +
                "/reports/SparkReport_BlazeDemo.html";

            // 2. Spark Reporter initialize
            ExtentSparkReporter sparkReporter = new ExtentSparkReporter(reportPath);

```

```

// 3. Report Dashboard and Look/Feel design
sparkReporter.config().setDocumentTitle("BlazeDemo Automation Report");
sparkReporter.config().setReportName("End-to-End Flight Booking Execution");
sparkReporter.config().setTheme(Theme.DARK);

// 4. Attach the spark in Extent Reports engine
extent = new ExtentReports();
extent.attachReporter(sparkReporter);

// 5. System/Environment details add
extent.setSystemInfo("Project", "BlazeDemo Capstone");
extent.setSystemInfo("SDET Engineer", "Harsh Choudhary");
extent.setSystemInfo("Operating System", System.getProperty("os.name"));
extent.setSystemInfo("Java Version", System.getProperty("java.version"));
}
return extent;
}
}

```

WaitUtils.java

```

package utils;

import base.DriverFactory;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public class WaitUtils {

    // Default timeout duration (10 seconds)
    private static final int DEFAULT_TIMEOUT = 10;

    private static WebDriverWait getWait() {
        return new WebDriverWait(DriverFactory.getDriver(),
Duration.ofSeconds(DEFAULT_TIMEOUT));
    }

    private static WebDriverWait getWait(int timeoutInSeconds) {

```

```

        return new WebDriverWait(DriverFactory.getDriver(),
Duration.ofSeconds(timeoutInSeconds));
    }

    /**
     * Waits for an element to be completely visible on the DOM and screen.
     */
    public static WebElement waitForElementVisible(By locator) {
        return getWait().until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    /**
     * Waits for an element to be visible with a custom timeout.
     */
    public static WebElement waitForElementVisible(By locator, int timeoutInSeconds) {
        return
getWait(timeoutInSeconds).until(ExpectedConditions.visibilityOfElementLocated(locator)
);
    }

    /**
     * Waits until the element is clickable (visible and enabled).
     */
    public static WebElement waitForElementClickable(By locator) {
        return getWait().until(ExpectedConditions.elementToBeClickable(locator));
    }

    /**
     * Checks if an element is present in the DOM (even if hidden).
     */
    public static WebElement waitForElementPresence(By locator) {
        return getWait().until(ExpectedConditions.presenceOfElementLocated(locator));
    }

    /**
     * Waits for the complete page to load using JavaScript Document Ready State.
     */
    public static void waitForPageLoad() {
        WebDriverWait wait = new WebDriverWait(DriverFactory.getDriver(),
Duration.ofSeconds(20));
        wait.until(driver -> String
            .valueOf(((JavascriptExecutor) driver).executeScript("return
document.readyState")))
            .equals("complete"));
    }

```



```
}  
}
```

-----src/test/resources-----

1.config/config.properties

browser=chrome

url=https://blazedemo.com/

2. testdata/BlazeDemoData

A	B	C	D	E	F	G	H	I	J	K	L
Name	Address	City	State	ZipCode	CardType	CardNumber	Month	Year	NameOnCard	Departure	Destination
Harsh Choudhary	123 Tech Park	Noida	Uttar Pradesh	201301	Visa	1.11122E+15	11	2026	Harsh Choudhary	Paris	London
Aman Singh	456 Developer Lane	Mumbai	Maharashtra	400001	American Express	5.55567E+15	12	2027	Aman Singh	Boston	Berlin
Priya Sharma	789 IT Hub	Bangalore	Karnataka	560001	Diner's Club	9.999E+15	5	2028	Priya Sharma	Portland	Rome

-----Dockerfile-----

FROM selenium/standalone-chrome:latest

USER root

WORKDIR /app

RUN apt-get update && apt-get install -y maven openjdk-25-jdk

COPY . .

RUN mvn clean install -DskipTests

CMD ["mvn", "test", "-DsuiteXmlFile=testng.xml"]

-----Jenkinsfile-----

```
pipeline {  
  agent any  
  
  tools {  
    maven 'Maven_3'  
  }  
  
  stages {  
    stage('Checkout Code') {  
      steps {  

```

```

        git branch: 'master',
        url: 'https://github.com/Harsh-Panghal/capstone_project'
    }
}

stage('Clean & Build') {
    steps {
        dir('BlazeDemo') {
            echo 'Cleaning old files and compiling the framework...'
            bat 'mvn clean compile'
        }
    }
}

stage('Execute TestNG Suite') {
    steps {
        dir('BlazeDemo') {
            echo 'Triggering the End-to-End Flight Booking Automation...'
            bat 'mvn test'
        }
    }
}

stage('Docker Build') {
    steps {
        dir('BlazeDemo') {
            bat 'docker build -t blazedemo-framework .'
        }
    }
}

stage('Docker Run') {
    steps {
        bat 'docker run --rm blazedemo-framework'
    }
}

post {
    always {
        echo 'Archiving Extent Reports & Screenshots for Dashboard...'
        archiveArtifacts artifacts: 'reports/**/*', allowEmptyArchive: true
    }
    success {
        echo 'E2E Automation Pipeline Completed Successfully!'
    }
}

```

```

    }
    failure {
        echo 'Pipeline Failed. Check the Extent Report and Screenshots for details.'
    }
}
}
}

```

-----pom.xml-----

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>BlazeDemo_Capstone_Project</groupId>
    <artifactId>BlazeDemo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.4</version>
                <configuration>
                    <testFailureIgnore>true</testFailureIgnore>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>
                    </suiteXmlFiles>
                </configuration>
            </plugin>
        </plugins>
    </build>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.18.1</version> </dependency>

        <dependency>
            <groupId>org.testng</groupId>

```

```

        <artifactId>testng</artifactId>
        <version>7.9.0</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
        <version>5.2.3</version>
    </dependency>
    <dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>5.1.1</version>
    </dependency>
</dependencies>
</project>

-----testng.xml-----
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="BlazeDemo Capstone Suite" verbose="2">

    <listeners>
        <listener class-name="listeners.TestListener" />
    </listeners>

    <test name="End To End Flight Booking Validation" group-by-instances="true">
        <classes>
            <class name="tests.BlazeDemoE2ETest" />
        </classes>
    </test>

</suite>

-----testng-negative.xml-----
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

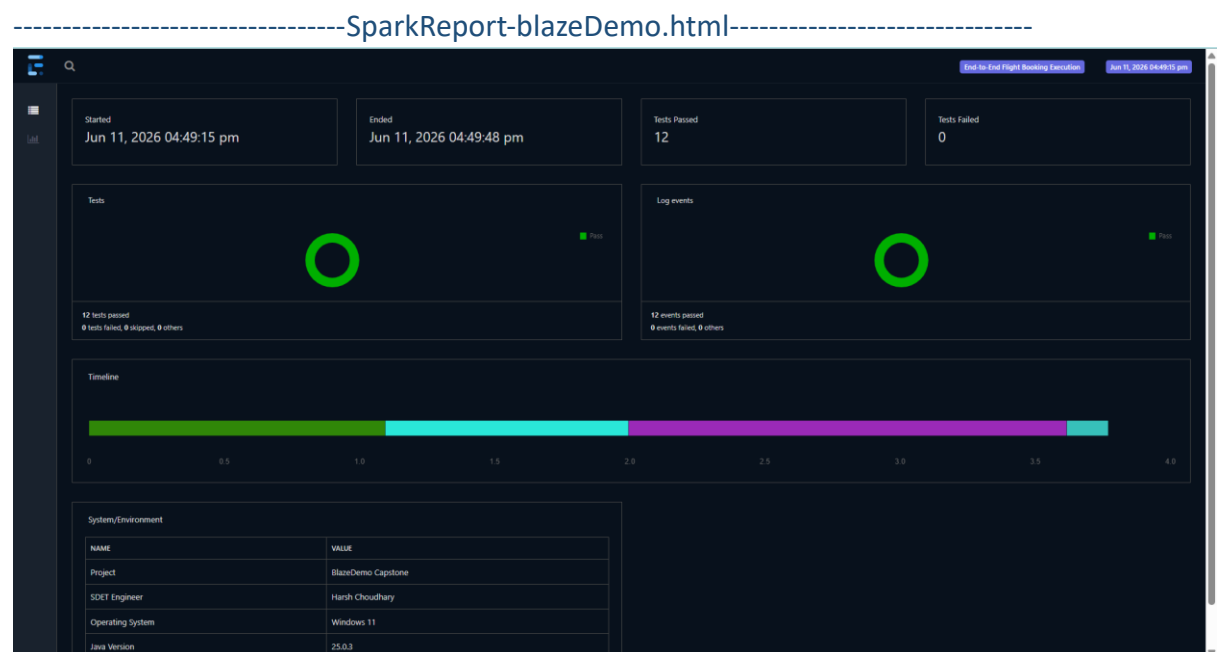
<suite name="BlazeDemo Negative Suite" verbose="2">

    <listeners>
        <listener class-name="listeners.TestListener" />
    </listeners>

```

```
<test name="Authentication and Validation Defect Tests">
  <classes>
    <class name="tests.BlazeDemoNegativeTests" />
  </classes>
</test>

</suite>
```



```
Test Passed: testVerifyBookingConfirmation
Browser closed successfully.
Test Suite Execution Finished! Generating Report...
PASSED: tests.BlazeDemoE2ETest.testFlightSearch
PASSED: tests.BlazeDemoE2ETest.testFlightSearch
PASSED: tests.BlazeDemoE2ETest.testFlightSearch
PASSED: tests.BlazeDemoE2ETest.testVerifyBookingConfirmation
PASSED: tests.BlazeDemoE2ETest.testVerifyBookingConfirmation
PASSED: tests.BlazeDemoE2ETest.testPurchaseFlight
PASSED: tests.BlazeDemoE2ETest.testVerifyBookingConfirmation
PASSED: tests.BlazeDemoE2ETest.testPurchaseFlight
PASSED: tests.BlazeDemoE2ETest.testSelectCheapestFlight
PASSED: tests.BlazeDemoE2ETest.testPurchaseFlight
PASSED: tests.BlazeDemoE2ETest.testSelectCheapestFlight
PASSED: tests.BlazeDemoE2ETest.testSelectCheapestFlight
```

```
=====
      End To End Flight Booking Validation
      Tests run: 12, Failures: 0, Skips: 0
=====
```

```
=====
BlazeDemo Capstone Suite
Total tests run: 12, Passes: 12, Failures: 0, Skips: 0
=====
```

The screenshot shows an IDE window titled 'Results of running suite'. The window has a search bar and a status bar indicating 'Passed: 12', 'Failed: 0', and 'Skipped: 0'. The main content area is divided into two panes: 'All Tests' and 'Summary'. The 'All Tests' pane shows a tree view of the test suite structure. The 'Summary' pane is currently empty.

Test Results:

- BlazeDemo Capstone Suite (12/0/0/0) (19.244 s)
 - End To End Flight Booking Validation (19.244 s)
 - tests.BlazeDemoE2ETest
 - testFlightSearch (1.809 s)
 - testSelectCheapestFlight (1.526 s)
 - testPurchaseFlight (3.861 s)
 - testVerifyBookingConfirmation (0.182 s)
 - testFlightSearch (1.312 s)
 - testSelectCheapestFlight (1.074 s)
 - testPurchaseFlight (3.354 s)
 - testVerifyBookingConfirmation (0.202 s)
 - testFlightSearch (1.119 s)
 - testSelectCheapestFlight (1.093 s)
 - testPurchaseFlight (3.507 s)
 - testVerifyBookingConfirmation (0.205 s)

Screenshots

Travel The World home

Flights from Paris to London:

Choose	Flight #	Airline	Departs: Paris	Arrives: London	Price
Choose This Flight	43	Virgin America	1:43 AM	9:45 PM	\$472.56
Choose This Flight	234	United Airlines	7:43 AM	10:45 PM	\$432.98
Choose This Flight	9696	Aer Lingus	5:27 AM	8:22 PM	\$200.98
Choose This Flight	12	Virgin America	11:23 AM	1:45 PM	\$765.32
Choose This Flight	4346	Lufthansa	1:45 AM	8:34 PM	\$233.98

Travel The World home

Your flight from TLV to SFO has been reserved.

Airline: United

Flight Number: UA954

Price: 400

Arbitrary Fees and Taxes: 514.76

Total Cost: **914.76**

Please submit the form below to purchase the flight.

Name

First Last

Address

123 Main St.

City

Anytown

State

State

Zip Code

12345

Card Type

Travel The World home

Thank you for your purchase today!

Id	1781164982194
Status	PendingCapture
Amount	555 USD
Card Number	xxxxxxxxxxxx1111
Expiration	11 /2018
Auth Code	888888
Date	Thu, 11 Jun 2026 08:03:02 +0000

```
{
  "id": "1781164982194",
  "status": "PendingCapture",
  "amount": "555",
  "currency": "USD",
  "authCode": "888888",
  "payment": {
    "cardNumber": "xxxxxxxxxxxx1111",
    "cardExpirationMonth": "11",
    "cardExpirationYear": "2018"
  }
}
```